

Algoritmi e Strutture Dati–parte I

lez. 16

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
(aula 11, Secondo Piano)
- Giovedì 9.00-11.00 (aula 11, Secondo Piano)

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Tipi di Archi

- Possiamo definire quattro tipi di archi in base alla foresta DF prodotta da una visita in profondità dal grafo G :
- **Archi d' albero**
- **Archi all' indietro**
- **Archi in Avanti**
- **Archi trasversali**

Archi d' Albero

- Sono gli archi nella foresta DF. L' arco (u, v) è un arco d' albero se v viene scoperto la prima volta durante l' esplorazione di (u, v) .

Archetti all' indietro

- Gli archetti all' indietro sono quegli archetti (u, v) che collegano un vertice u ad un antenato v in un albero DF.
- I cappi, che possono presentarsi nei grafi orientati, sono considerati archetti all' indietro.

Archi in avanti

- Gli archi in Avanti sono gli archi (u, v) (diversi dagli archi d'albero) che collegano un vertice u ad un discendente v in un albero DF.

Archivi trasversali

- Gli archivi trasversali sono tutti gli altri archivi.
- Possono connettere i vertici nello stesso albero DF, purché un vertice non sia antenato dell'altro, oppure possono connettere vertici di alberi DF differenti.

Colore degli Archi

- L' algoritmo DFS possiede le informazioni necessarie per classificare gli archi che incontra.
- L' idea chiave è che ogni arco (u, v) può essere classificato in base al colore del vertice v che viene raggiunto quando l' arco viene ispezionato la prima volta:
 - **WHITE**: indica un **arco d' albero**
 - **GRAY**: indica un **arco all' indietro**
 - **BLACK**: indica un **arco in avanti** o **trasversale**

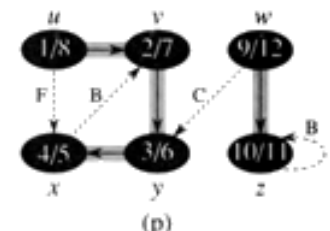
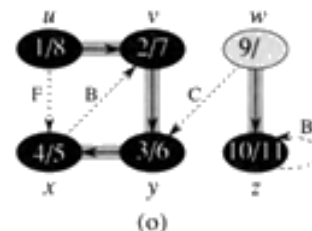
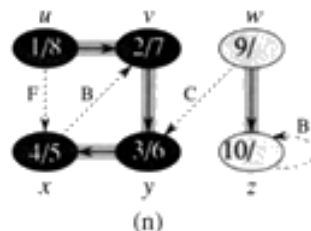
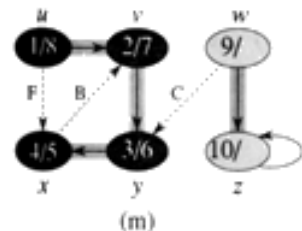
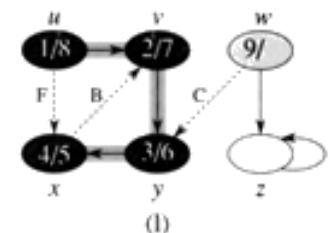
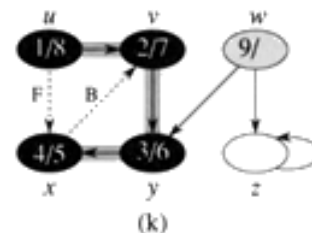
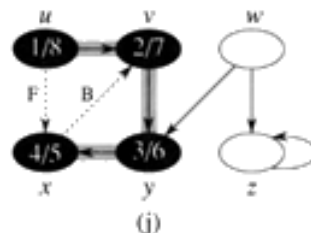
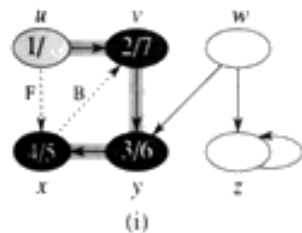
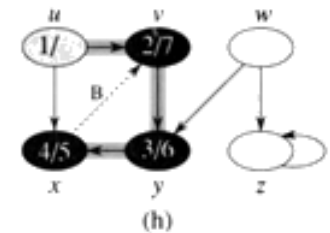
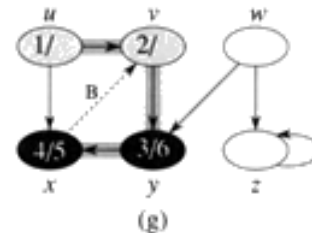
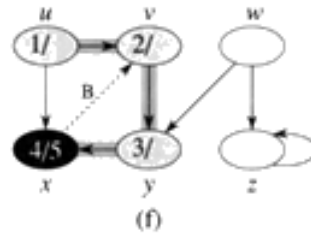
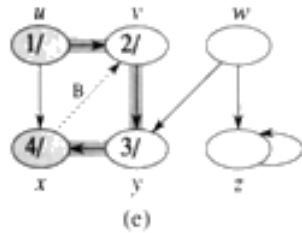
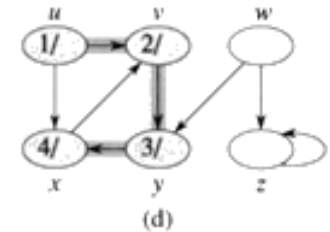
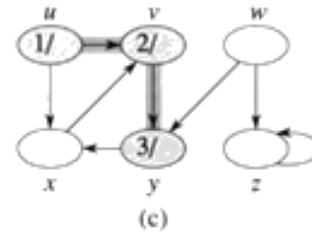
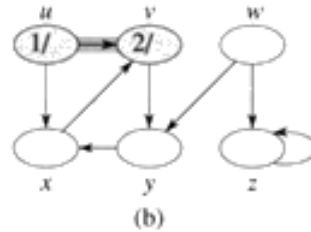
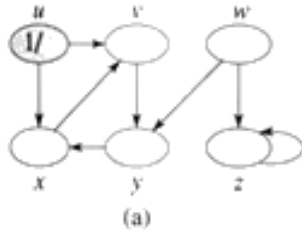
Dimostrazione

- Il primo caso è immediato dalla specifica dell' algoritmo.
- Per il secondo caso, I vertici grigi formano sempre una catena lineare di discendenti che corrisponde alle chiamate attive della DFS_VISIT. L' ispezione dell' arco parte sempre dal vertice grigio più profondo, quindi **un arco, partente da un vertice grigio, che raggiunge un altro vertice grigio, raggiunge un antenato.**

(cont.)

- Il terzo caso gestisce la terza possibilità; si può dimostrare che tale arco (u, v) è un arco in avanti se $d[u] < d[v]$ ed un arco trasversale se $d[u] > d[v]$

DFS



Teorema

- In una visita in profondità in un grafo non orientato G , gli archi di G possono essere archi d'albero o archi all'indietro.

Dimostrazione

- Sia (u, v) un arco arbitrario di G e supponiamo, senza perdere in generalità, che $d[u] < d[v]$. Allora, il vertice v deve essere scoperto e completato prima che sia completato u (mentre u è grigio), perchè v si trova nella lista di adiacenza di u .
- Se l' arco (u, v) viene ispezionato prima nella direzione da u a v allora v è un vertice non scoperto (bianco) e l' arco è d' albero.

(cont.)

- Se l' arco(u, v) viene ispezionato prima nella direzione da v a u allora u è un vertice grigio e l' arco è all' indietro.

Connessione di un grafo

- In un grafo non orientato $G=(V,E)$, due vertici u e v sono **connessi (connected)** se in G esiste un **cammino tra u e v** . Altrimenti, essi sono **non connessi (disconnected)**.
- Un **grafo è connesso** se qualunque coppia di vertici u e v distinti del grafo è connessa.

(cont.)

- Una **componente connessa** (**connected component**) di un grafo G è un subgrafo, $H(V^1, E^1)$ $V^1 \subseteq V$ $E^1 \subseteq E$, connesso massimo di G . Ciascun vertice appartiene esattamente ad una ed una sola componente connessa. Allo stesso modo ciascun arco appartiene esattamente ad una ed una sola componente connessa.

Grafo Debolmente Connesso

- Un grafo orientato (directed graph) è **debolmente connesso** (**weakly connected**) se rimpiazzando tutti gli archi orientati con archi non orientati si ottiene un **grafo connesso** (undirected graph).

Grafo Fortemente Connesso

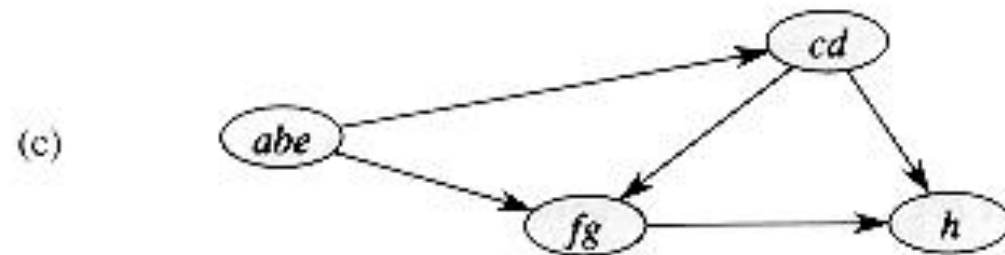
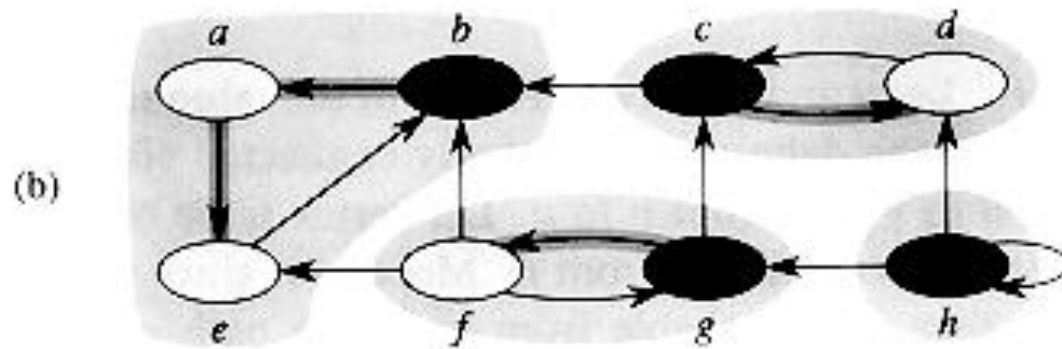
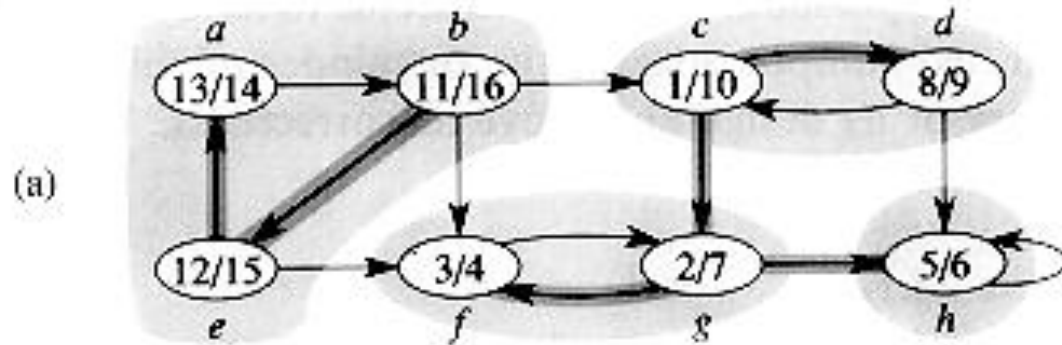
- Un grafo è **fortemente connesso** (**strongly connected**) se contiene un cammino orientato da u a v per ogni coppia di vertici u, v . Le componenti forti (**strong components**) di un grafo sono i subgrafi fortemente connessi massimi.

Calcolo delle Componenti fortemente connesse

- L' algoritmo per trovare le componenti fortemente connesse di un grafo $G = (V, E)$ utilizza il grafo trasposto di G , ossia il grafo $G^T = (V, E^T)$, dove $E^T = \{(u, v) : (v, u) \in E\}$
- L' algoritmo seguente calcola in tempo $\Theta(V + E)$ le componenti connesse.

STRONGLY_CONNECTED_COMPONENTS(G)

- Chiama $DFS(G)$ per calcolare i tempi di completamento $u.f$ per ciascun vertice u
- Calcola G^T
- Chiama $DFS(G^T)$, ma nel ciclo principale di DFS, considera i vertici in tempo decrescente rispetto ai tempi calcolati nella riga 1
- Genera l' output dei vertici di ciascun albero della foresta DF che è stata prodotta nella riga 3 come una singola componente fortemente connessa.



Connettività di un grafo

- Un **cut** (o **vertex cut**) di un grafo connesso G è l'**insieme dei vertici la cui rimozione rende G disconnesso**.
- La **connettività (connectivity)** $\kappa(G)$ è la **cardinalità del più piccolo vertex cut**. Un grafo è chiamato **k -connected** se la **connettività dei suoi vertici è k** o più grande. Un vertex cut per due vertici u e v è un insieme di vertici la cui rimozione disconnette u e v .

(cont.)

- La **local connectivity** $\kappa(u, v)$ è la **cardinalità del più piccolo vertex cut che separa u e v** . La local connectivity è simmetrica per grafi non orientati, i.e. $\kappa(u, v) = \kappa(v, u)$. $\kappa(G)$ è il minimo dei $\kappa(u, v)$ su tutte le coppie di vertici u, v .

Edge Cut

- Un **edge cut** di un grafo G è l'insieme degli archi (edges) la cui rimozione rende il grafo non connesso. L'**edge-connectivity** $\kappa'(G)$ è la cardinalità del più piccolo edge cut, e la **local edge-connectivity** $\kappa'(u,v)$ di due vertici u,v è la cardinalità del più piccolo edge cut che disconnette u da v . Un grafo è **k -edge-connected** se la sua edge connectivity è $\geq k$.

Insiemi Disgiunti

- Una struttura dati per insiemi disgiunti dinamici mantiene una collezione $S = \{S_1, S_2, \dots, S_k\}$.
- Ciascun insieme è identificato da un **rappresentante**, che è un elemento dell'insieme.

(cont.)

- In alcune applicazioni non è importante quale elemento sarà utilizzato come rappresentante, basta che se lo si richiede due volte, senza modificare l'insieme, si ottenga sempre lo stesso valore.
- In altre applicazioni sarà fissato con una regola prestabilita

Operazioni su Insiemi Disgiunti

- Ogni elemento di un insieme disgiunto è rappresentato da un oggetto. Indicando con x un oggetto, vogliamo supportare le seguenti operazioni:
 - **MAKE_SET(x)**
 - **UNION(x, y)**
 - **FIND_SET(x)**

MAKE_SET(x)

- Crea un nuovo insieme il cui unico elemento (e rappresentante) è x .
- Poichè gli insiemi sono disgiunti, x non può trovarsi in qualche altro insieme.

UNION(x, y)

- UNION(x, y) unisce gli insiemi dinamici S_x e S_y che contengono x e y in un nuovo insieme che è l' unione di questi due insiemi.
- Si suppone che i due insiemi siano disgiunti prima dell' operazione. Il rappresentante dell' insieme risultante è un elemento qualsiasi di $S_x \cup S_y$.
- Poichè richiediamo che gli insiemi della collezione siano disgiunti, eliminiamo S_x e S_y dalla collezione S

FIND_SET(x)

- FIND_SET(x) restituisce un puntatore al rappresentante dell'insieme che contiene x .

Calcolo delle Componenti Connesse mediante insiemi disgiunti

- La procedura `CONNECTED_COMPONENTS` usa le operazioni degli insiemi disgiunti per calcolare le componenti connesse di un grafo.
- Una volta che `CONNECTED_COMPONENTS` ha preelaborato il grafo, la procedura `SAME_COMPONENT` è in grado di determinare se due vertici sono nella stessa componente connessa.

CONNECTED_COMPONENTS(G)

1. **for** ogni vertice $v \in G.V$
2. $MAKE_SET(v)$
3. **for** ogni arco $(u, v) \in G.E$
4. **if** $FIND_{SET}(u) \neq FIND_{SET}(v)$
5. $UNION(u, v)$

SAME_COMPONENT(u, v)

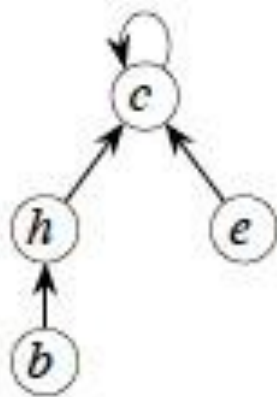
1. **if** $FIND_SET(u) = FIND_SET(v)$
2. **return** TRUE
3. **else return** FALSE

Foreste di Insiemi Disgiunti

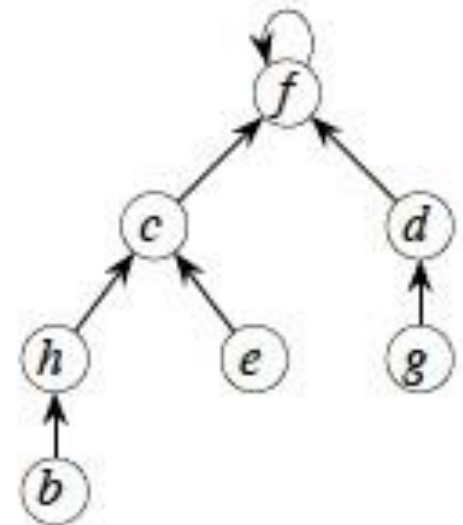
- Possiamo rappresentare gli insiemi disgiunti con alberi radicati, dove ogni nodo contiene un elemento ed ogni albero rappresenta un insieme.
- In una foresta di Insiemi Disgiunti, **ogni elemento punta soltanto a suo padre.**
- Il nodo radice di ogni albero contiene il rappresentante ed è padre di sè stesso.

Operazioni su Foreste di Insiemi Disgiunti

- Un' operazione **MAKE_SET** crea semplicemente **un albero con un solo nodo**.
- Un' operazione **FIND_SET** segue i puntatori ai **padri finchè non trova la radice dell' albero**. I nodi visitati in questo cammino semplice verso la radice costituiscono il **cammino di radice**.
- Un' operazione **UNION** fa sì che la radice di un albero punti alla radice dell' altro albero.



UNION(e, g)



Euristiche per Insiemi Disgiunti

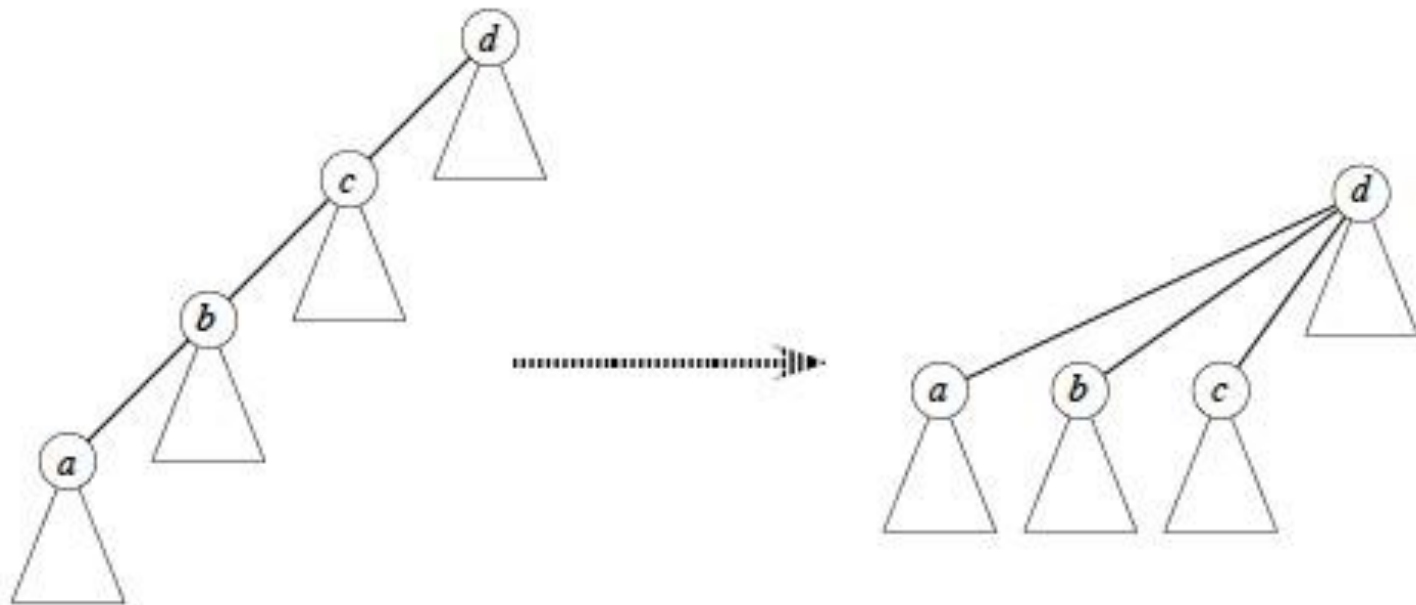
- Una sequenza di $n - 1$ operazioni UNION può creare un albero che è una catena lineare di n nodi. Utilizzando due euristiche possiamo ottenere un tempo di esecuzione che è quasi lineare nel numero totale di operazioni m

Unione per rango

- L'idea dell'unione per rango consiste nel fare in modo che la radice dell'albero con meno nodi punti alla radice dell'albero con più nodi.
- Per ogni nodo manteniamo un **rango che è limite superiore per l'altezza del nodo**.
- Nell'unione per rango, **la radice con il rango più piccolo viene fatta puntare alla radice con il rango più grande** durante un'operazione UNION.

Unione per Compressione del Cammino

- L' euristica compressione per cammino viene utilizzata durante le operazioni FIND_SET per fare in modo che ciascun nodo nel cammino di ricerca punti direttamente alla radice.
- La compressione del cammino non cambia i ranghi.



Implementazione dell' unione per rango

- Per l' implementazione dell' unione per rango bisogna tener traccia dei ranghi.
- Per ogni nodo x manteniamo il valore $x.rank$, che è un limite superiore per l' altezza di x .
- Quando l'operazione MAKE_SET crea un insieme con un solo elemento, il rango iniziale dell' unico nodo nel corrispondente albero è 0.

(cont.)

- Quando applichiamo UNION a due alberi, si presentano due casi, a seconda che le radici abbiano oppure no lo stesso rango.
- **Se le radici non hanno lo stesso rango, trasformiamo la radice di un rango più alto nel padre della radice di rango più basso, ma i ranghi restano inalterati.**
- **Se le radici hanno lo stesso rango, trasformiamo arbitrariamente una delle radici in padre e ne incrementiamo il rango.**

(cont.)

- Indichiamo con $x.p$ il padre del nodo x . La procedura LINK riceve i puntatori a due radici come input.

MAKE_SET(x)

1. $x.p \leftarrow x$
2. $x.rank \leftarrow 0$

UNION(x, y)

1. *LINK(FINDSET(x), FINDSET(y))*

LINK(x, y)

1. **if** $x.rank > y.rank$
2. $y.p \leftarrow x$
3. **else** $x.p \leftrightarrow y$
4. **if** $x.rank = y.rank$
5. $y.rank = y.rank + 1$

FINDSET(x)

1. **if** $x \neq x.p$
2. $x.p \leftarrow \text{FINDSET}(x.p)$
3. **return** $x.p$

TEOREMA

- Una sequenza di m operazioni MAKE_SET, UNION, FINDSET, n delle quali sono operazioni MAKE_SET, può essere eseguita, nel caso peggiore, in una foresta di insiemi disgiunti con l'unione per rango e la compressione del cammino nel tempo $O(m \alpha(n))$.

Ackerman function

- Nel 1920 Ackerman pubblicò una funzione che cresceva più rapidamente rispetto a quelle che erano note a quel tempo.
- La funzione di Ackerman originaria aveva tre argomenti
- A partire dalla funzione originaria di Ackerman, ne sono state proposte simili.

Funzione Ackerman-like

- Per $k \geq 0$ e $j \geq 1$ interi qualsiasi, definiamo la funzione $A_k(j)$ in questo modo

$$A_k(j) = \begin{cases} j + 1 & \text{se } k = 0 \\ A_{k-1}^{(j+1)}(j) & \text{se } k \geq 1 \end{cases}$$

- Valgono:

- $A_{k-1}^{(0)}(j) = j$

- $A_{k-1}^{(i)}(j) = A_{k-1} \left(A_{k-1}^{(i-1)}(j) \right)$

Proprietà della Funzione

- Per ogni intero $j \geq 1$, $A_1(j) = 2j + 1$
- Per ogni intero $j \geq 1$, $A_2(j) = 2^{j+1}(j + 1) - 1$
- $A_k(j) = \begin{cases} j + 1 & \text{se } k = 0 \\ A_{k-1}^{(j+1)}(j) & \text{se } k \geq 1 \end{cases}$
- Quindi:
 - $A_0(1) = 1 + 1 = 2$
 - $A_1(1) = 2 * 1 + 1 = 3$
 - $A_2(1) = 2^2(2) - 1 = 7$
 - $A_3(1) = A_2^2(1) = A_2(A_2(1)) = A_2(7) = 2^8(8) - 1 = 2^{11} - 1 = 2047$

(cont.)

- $A_k(j) = \begin{cases} j + 1 & \text{se } k = 0 \\ A_{k-1}^{(j+1)}(j) & \text{se } k \geq 1 \end{cases}$
- $A_4(1) = A_3^2(1) = A_3(A_3(1)) = A_3(2047) = A_2^{2048}(2047) \gg A_2(2047)$
- $A_2(j) = 2^{j+1}(j + 1) - 1$
- $A_4(1) \gg 2^{2048}(2048) - 1 \gg 2^{2048}(2^{11}) - 1$
- $A_4(1) \gg 2^{2059} - 1 \gg 2^{2058} \gg 2^{2000} 2^{58}$
- $(2^{10} > 10^3)$
- $A_4(1) \gg (2^{10})^{200} (2^{10})^5 2^8 \gg 10^{600} 10^{15} 256$
- $A_4(1) \gg 256 * 10^{615}$

Inverso della Funzione di Ackerman

- Definiamo l'inverso della funzione $A_k(n)$, per qualsiasi intero $n \geq 0$, in questo modo:
 $\alpha(n) = \min\{k: A_k(1) \geq n\}$
- Ovvero $\alpha(n)$ è il livello k più basso per il quale $A_k(1)$ è almeno pari a n . Abbiamo che:

- $$\alpha(n) = \begin{cases} 0 & \text{per } 0 \leq n \leq 2 \\ 1 & \text{per } n = 3 \\ 2 & \text{per } 4 \leq n \leq 7 \\ 3 & \text{per } 8 \leq n \leq 2047 \\ 4 & \text{per } 2048 \leq n \leq A_4(1) \gg 256 * 10^{615} \end{cases}$$

(cont.)

- È soltanto per quei valori eccezionalmente grandi di n che $\alpha(n) > 4$.
- Quindi possiamo ritenere che $\alpha(n) \leq 4$ in tutte le applicazioni pratiche.